

BCC2001

— ORIGIN
'warn'.

monitor

thing 2 a
duties 3 a
picture 3 a

Diego Bertolini

diegobertolini@gmail.com

Aula Passada

- Funções ;
- Atividades Utilizando funções ;

Aula de Hoje

- A compilação do Programa ;
- Argumentos para função main()
- Bibliotecas (stdio.h, stdlib.h, math.h)
- Criando uma biblioteca ;
- Trabalhando com vários arquivos em C;
- Ideia de Recursividade;

Argumentos na linha de comando

- Vimos que a clausula `main()` indica a função principal do programa ;
- Ela é responsável pelo início da execução do programa ;
- Quando aprendemos a criar nossas próprias funções, vimos que era possível passar uma lista de parâmetros ;

Argumentos na linha de comando

- A função `main()` também é uma função. Portanto, ela também pode receber uma lista de parâmetros no início da execução do programa.
- Para receber esses parâmetros a função `main` adquire a seguinte forma:

Argumentos na linha de comando

```
1  #include <stdio.h>
2
3  int main(int argc, char *argv)
4  {
5
6      return 0;
7  }
8
```


Argumentos na linha de comando

Note que agora a função `main()` recebe dois parâmetros de entrada:

int argc : Trata-se de um número inteiro que indica o número de parâmetros com os quais a função `main` foi chamada na linha de comando ;

Argumentos na linha de comando

O valor de **argc** é sempre maior ou igual a 1. Esse parâmetro vale 1 se o programa foi chamado sem nenhum parâmetro (o nome do programa é contado como um argumento da função), e é somado +1 em **argc** para cada parâmetro passado para o programa.

Argumentos na linha de comando

char *argv[] : Trata-se de um ponteiro para uma matriz de strings. Cada uma das strings contidas nessa matriz é um dos parâmetros com os quais a função main foi chamada na linha de comando. Ao todo, existem **argc** strings guardadas em argv.

Argumentos na linha de comando

A string guardada em `argv[0]` sempre aponta para o nome do programa (lembre-se o nome do programa é contado como argumento da função)

Argumentos na linha de comando

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(int argc, char *argv[])
5  {
6      printf("Numero de Parâmetros de Entrada: %d\n", argc);
7      printf("Parâmetros de Entrada: %s\n", argv[1]);
8      printf("Parâmetros de Entrada: %s\n", argv[2]);
9      printf("Parâmetros de Entrada: %s\n", argv[3]);
10     return 0;
11 }
12
```

Argumentos na linha de comando

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(int argc, char *argv[])
5  {
6      if (argc == 1)
7          printf("Nenhum parâmetro passado para o programa %s \n", argv[0]);
8      else{
9          int i ;
10         for (i = 1; i < argc ; i++) {
11             printf("Parâmetro %d de Entrada: %s \n",i, argv[i]);
12         }
13     }
14     return 0;
15 }
16 |
```

Argumentos na linha de comando

Perceba que os argumentos de entrada são strings ;

Podemos utilizar as funções `atoi` e `atof` para converter strings em um inteiro ou ponto flutuante.

Argumentos na linha de comando

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(int argc, char *argv[])
5  {
6      printf("Numero de Parâmetros de Entrada: %d\n", argc);
7
8      float p1 ;
9      int p2 ;
10
11      p1 = atof(argv[1]) ;
12      p2 = atoi(argv[2]) ;
13
14      printf("Float = %f\n", p1 / 3) ;
15      printf("Inteiro = %d\n", p2*9) ;
16      printf("String = %s\n", argv[3]) ;
17
18      return 0;
19  }
```


Atividade

Faça um programa que receba por linha de comando 2 parâmetros que serão um Valor_Inicial e um Valor_Final, caso o valor final seja menor que o valor inicial, imprima que os valores estão incorretos, caso correto, imprima a soma dos números ímpares compreendidos entre os dois (inclusive).

Algumas Bibliotecas e Funções úteis

Biblioteca de Entrada e Saída: stdlib.h

int atoi(const char *) : converte string em inteiro;
double atof(const char *) : converte string em double;
int rand(void) : Gera Números aleatórios ;
int abs(int): valor absoluto ;

Algumas Bibliotecas e Funções úteis

Biblioteca de Funções Matemáticas: math.h

double cos(double) : calcula o cosseno de um ângulo em radianos ;

double sin(double) : calcula o seno de um ângulo em radianos ;

double tan(double) : calcula a tangente de um ângulo em radianos ;

double exp(double) : função exponencial ;

double log(double) : logaritmo Natural ;

double log10(double) : logaritmo comum (base 10) ;

CONSTANTES:

M_PI: VALOR DE PI

Algumas Bibliotecas e Funções úteis

Biblioteca de Funções Matemáticas: math.h

double pow(double, double) : retorna a base elevada ao expoente ;

double sqrt(double) : retorna a raiz quadrada de um número ;

FUNÇÕES DE ARREDONDAMENTO

double ceil(double) : arredonda para cima um número ;

double floor(double) : arredonda para baixo um número ;

Atividades

■ Faça um programa que leia 5 números por linha de comando, calcule e imprima, na ordem em que os números foram digitados:

- 1) Raiz Quadrada ;
- 2) O número elevado a 5 ;
- 3) Um número com três casas decimais arredondado para baixo;
- 4) Um número com três casas decimais arredondado para cima;
- 5) O valor absoluto de um número ;

Atividades

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4
5  int main(int argc, char *argv[])
6  {
7      printf("Numero de Parâmetros de Entrada: %d\n", argc);
8
9      double p1, p2, p3, p4, p5;
10
11     p1 = atof(argv[1]) ;
12     p2 = atof(argv[2]) ;
13     p3 = atof(argv[3]) ;
14     p4 = atof(argv[4]) ;
15     p5 = atof(argv[5]) ;
16
17     printf("Raiz = %lf\n", sqrt(p1)) ;
18     printf("Potência = %lf\n", pow(p2, 5)) ;
19     printf("Arred. para Baixo = %lf\n", floor(p3)) ;
20     printf("Arred. para Cima = %lf\n", ceil(p3)) ;
21     printf("Log na base 10 = %lf\n", log10(p4)) ;
22     printf("Log na base 2 = %lf\n", log(p4)) ;
23     printf("Valor Absoluto = %d \n", abs(p5)) ;
24
25     return 0;
26 }
```

Criando Bibliotecas.

- De modo geral, os arquivos de bibliotecas na linguagem C são terminados com a extensão **.h**
- Incluiremos novas bibliotecas através do **#include**
- Usamos `< >` para incluir bibliotecas que são próprias do sistema (stdio.h e stdlib.h)
- Usaremos `" "` para incluir bibliotecas que desenvolveremos . Estas deveram estar no mesmo diretório onde se encontra o programa, ou então especificar a localidade.

Criando Bibliotecas.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4
5  #include "aritmetica.h"
6  #include "\\home\\diego\\Codigos\\C\\BCC\\soma.h"
7  #include "C:\\Programas\\soma.h"
8
```

Criando Bibliotecas.

- Uma biblioteca é como seu arquivo de código-fonte principal, com a diferença de que ele não possui a função `main()`. Isso ocorre porque o seu programa não vai começar na biblioteca.

Criando Bibliotecas.

- Qual a vantagem de criarmos bibliotecas ?
- Imaginemos algo que usamos sempre, poderíamos criar uma biblioteca, e chamá-la quando precisarmos realizar aquilo.
- As quatro operações básicas da matemática (adição, produto, subtração, divisão) ?
- Se implementarmos uma biblioteca, que quando fizéssemos a chamada a **soma** passando dois números ele nos retornasse a soma destes? Ou a divisão, ou o produto, a subtração....

Criando Bibliotecas.

- Precisaremos de dois (2) arquivos:
 - **Cabeçalho (ou header) da biblioteca:** Esse arquivo contém as declarações e definições do que está contido dentro da biblioteca. Aqui são definidas quais funções (apenas seu protótipo), tipos e variáveis que farão parte da biblioteca. Sua extensão é **.h**
 - **Código-fonte da biblioteca:** O arquivo contém a implementação das funções definidas no cabeçalho. Sua extensão é **.c**

Criando Bibliotecas.

aritmetica.h ✕

```
1 float soma (float a, float b) ;  
2 float subtrai (float a, float b) ;  
3 float divide (float a, float b) ;  
4 float multiplica (float a, float b) ;  
5
```

Criando Bibliotecas.

- Sempre deveremos incluir nossa própria biblioteca no código-fonte dela. Para isso usaremos o `#include "nome_da_biblioteca.h"`

```
1  #include "aritmetica.h"
2
3  float soma (float a, float b){
4      return a+b ;
5  }
6
7  float subtrai (float a, float b){
8      return a-b ;
9  }
10
11 float divide (float a, float b){
12     return a/b ;
13 }
14
15 float multiplica (float a, float b){
16     return a*b ;
17 }
```

Criando Bibliotecas.

- Por fim, temos que incluir nossa biblioteca dentro do programa.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "aritmetica.h"
4
5  int main()
6  {
7      float r ;
8
9      r = soma(2, 2.5);
10     printf("Soma: %f \n", r);
11
12     r = subtrai(10, 2.5);
13     printf("Subtrai: %f \n", r);
14
15     r = multiplica(10, 2.5);
16     printf("Multiplica: %f \n", r);
17
18     r = divide(10, 2.5);
19     printf("Divide: %f \n", r);
20
21     return 0;
22 }
```


Atividade

Faça uma biblioteca, que retorne:

01: Faça uma função que calcule e retorne o número neperiano. Passar o parâmetro N para a função.

$$\frac{1}{0!} + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \dots + \frac{1}{N!}$$

02: O super fatorial de um número N é definida pelo produto dos N primeiros fatoriais de N . Assim, o super fatorial de 4 é $\text{sf}(4) = 1! * 2! * 3! * 4! = 288$. Passar parâmetro para a função e retornar o valor de sf .

03: Escreva uma função que receba um número inteiro positivo n . Calcule e retorne o somatório de 1 até n : $1 + 2 + 3 + \dots + n$.

Funções Recursivas

thing-2
duties 2 + 1
picture 2 + 1
screen 2 + 1

Dúvida

Como escrever os números de
1 a 5 na tela?

Solução?

```
int main(void) {  
    printf("1 2 3 4 5");  
    return 0;  
}
```

```
int main(void) {  
    printf("1\n 2\n 3\n 4\n 5\n");  
    return 0;  
}
```

Talvez uma função possa ser usada, é possível?

Talvez uma função possa ser usada, é possível?

```
int escreva(int n) {  
    int i;  
    for (i = 1; i <= n; i++)  
        printf("%d\n", i);  
}
```

Talvez uma função possa ser usada, é possível?

Talvez uma função possa ser usada, é possível?

```
int escreva(int n) {  
    printf("%d\n", n);  
}
```

Recursividade

Um objeto é dito recursivo se ele consistir parcialmente ou for definido em termos de si mesmo;

Em termos de programação: a recursividade é uma técnica em que uma rotina (ou módulo), no processo de execução de suas tarefas, chama a si mesma;

Recursividade...



Talvez uma função possa ser usada, é possível?

Talvez uma função possa ser usada, é possível?

```
int escreva(int n) {  
    printf("%d\n", n);  
    escreva(n-1);  
}
```

Talvez uma função possa ser usada, é possível?

Quando as chamadas a escreva (n-1) irão parar?

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 int escreva (int n)
5 {
6     printf("%d\n", n);
7     escreva(n-1);
8 }
9
10 int main()
11 {
12     escreva(10);
13     system("PAUSE");
14 }
```

```
29736
-29737
-29738
-29739
-29740
-29741
-29742
-29743
-29744
-29745
-29746
-29747
-29748
-29749
-29750
-29751
-29752
-29753
-29754
-29755
-29756
-29757
-29758
-29759
-29760
```

Recursividade

- A recursividade (ou recursão) é uma técnica poderosa em definições matemáticas;
- A recursividade em programação se dá através de módulos de programa (funções, procedimentos, sub-rotinas..);
- Assim pode-se dar um nome a uma função; Função essa que pode chamar a si própria através deste nome;

Conceitos Importantes!

- Procedimentos recursivos exigem uma condição de parada, pois existe a possibilidade das iterações não terminarem.
- Uma exigência fundamental é que a chamada recursiva a um procedimento P esteja sujeita a uma condição B, a qual se torna satisfeita em algum momento da computação.

Exemplo

- Nosso primeiro programa não tem uma condição de parada!

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 int escreva (int n)
5 {
6     printf("%d\n", n);
7     escreva(n-1);
8 }
9
10 int main()
11 {
12     escreva(10);
13     system("PAUSE");
14 }
```

```
-29736
-29737
-29738
-29739
-29740
-29741
-29742
-29743
-29744
-29745
-29746
-29747
-29748
-29749
-29750
-29751
-29752
-29753
-29754
-29755
-29756
-29757
-29758
-29759
-29760
```

Exemplo

- Agora com condição de Parada!

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 int escreva (int n)
5 {
6     if (n==0)
7         return;
8     else
9     {
10         printf("%d\n", n);
11         escreva(n-1);
12     }
13 }
14
15 int main()
16 {
17     escreva(10);
18     system("PAUSE");
19 }
```

Condição de parada



A terminal window showing the output of the recursive function. The numbers 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, and 0 are printed on separate lines. Below the numbers, the text "Pressione qualquer tecla para continuar" is visible, which is partially cut off in the image.

```
10
9
8
7
6
5
4
3
2
1
0
Pressione qualquer tecla para continuar
```


Exemplo Clássico: Fatorial

$$0! = 1$$

$$n > 0: n! = n * (n-1)!$$

1) Solução Não Recursiva

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 int fatorial (int num)
5 {
6     int i, fat = 1 ;
7     for (i = 1; i <= num; i++)
8         fat = fat*i;
9     return (fat);
10 }
11
12 int main()
13 {
14     int valor;
15     printf("Digite um valor para o Calculo do Fatorial: ");
16     scanf("%d", &valor);
17
18     printf("Valor do fatorial: %d\n\n", fatorial(valor));
19
20     system("PAUSE");
21 }
```

```
Digite um valor para o Calculo do Fatorial: 5
Valor do fatorial: 120
Pressione qualquer tecla para continuar. . .
```

Exemplo Clássico: Fatorial

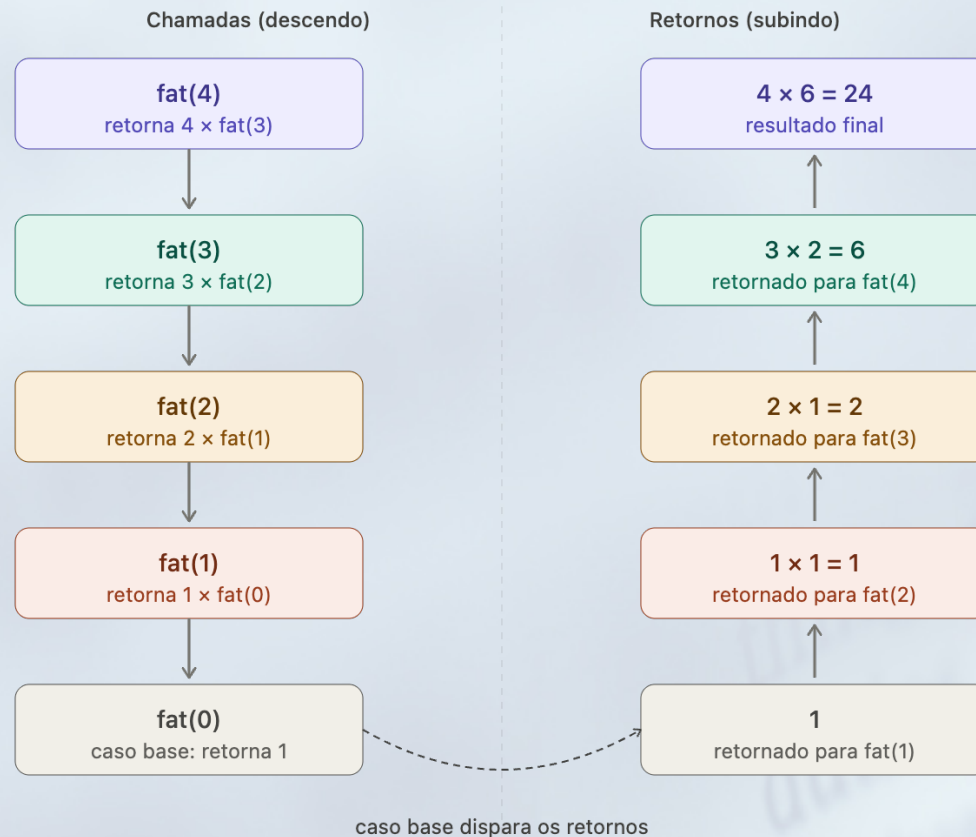
Solução Recursiva

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 int fatorial (int n)
5 {
6     if (n == 0) return(1);
7     else
8         if (n >0) return (n * fatorial(n-1));
9 }
10
11 int main()
12 {
13     int valor;
14     printf("Digite um valor para o Calculo do Fatorial: ");
15     scanf("%d", &valor);
16
17     printf("Valor do fatorial: %d\n\n", fatorial(valor));
18
19     system("PAUSE");
20 }
```

```
Digite um valor para o Calculo do Fatorial: 5
Valor do fatorial: 120

Pressione qualquer tecla para continuar. . .
```

Simulação do Exemplo do Fatorial



Recursividade

Descida: cada chamada não consegue calcular nada ainda, porque depende do resultado da próxima. A função fica "suspensa" na pilha esperando.

Caso base: $\text{fat}(0)$ é o único que sabe a resposta sem precisar de ninguém. Ele dispara a volta.

Subida: cada função retorna seu resultado para quem a chamou, multiplicando pelo valor de n que estava guardado na pilha.

Lembre-se: A função **não some** quando chama a si mesma — ela fica pausada na pilha, guardando o valor de n , até receber a resposta de volta.

Quando não usar Recursividade?

- Quando existir uma solução iterativa melhor: ou por ter um complexidade de tempo ou espaço menor.
- Mesmo sendo o problema ou seus dados definidos em termos recursivos não há a garantia que o algoritmo recursivo seja o melhor caminho de resolução.

Quando não usar Recursividade?

- Soluções recursivas são apropriadas para problemas com natureza recursiva;
- Mesmo assim, não é garantido que, para estes casos a solução recursiva seja a melhor;

Fibonacci

- Números de Fibonacci é uma sequência definida por:

$$F(n) = \begin{cases} 0, & \text{se } n = 0; \\ 1, & \text{se } n = 1; \\ F(n-1) + F(n-2) & \text{outros casos.} \end{cases}$$

0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 ...

- Sequência descrita por Leonardo de Pisa ou Fibonacci.
- Para descrever o crescimento de uma população de coelhos

Fibonacci Iterativo

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 void fib(int n)
5 {
6     int i, atual = 0, ultimo = 0, penultimo = 1;
7     for (i = 1; i <= n; i++)
8     {
9         atual = ultimo + penultimo;
10        penultimo = ultimo;
11        ultimo = atual;
12        printf("%d ", atual);
13    }
14 }
15
16 int main()
17 {
18     int valor;
19     printf("Digite um valor para o Calculo de Fibonacci: ");
20     scanf("%d", &valor);
21
22     fib(valor);
23
24     system("PAUSE");
25 }
```

Fibonacci Recursivo

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 int fib(int n)
5 {
6     if (n <= 1)
7         return(n);
8     else
9         return fib(n-1) + fib(n-2);
10 }
11
12 int main()
13 {
14     int valor;
15     printf("Digite um valor para o Calculo de Fibonacci: ");
16     scanf("%d", &valor);
17
18     printf("%d \n\n", fib(valor));
19
20     system("PAUSE");
21 }
```

Cuidado!

- Cuidado: rotinas recursivas requerem área para armazenamento dos valores dos objetos a cada chamada;
- Isto acarreta num risco de exceder o espaço disponível e ocorrer uma falha no programa;

Só para termos uma ideia...

n	10	20	30	50	100
FibRec	8 ms	1 s	2 min	21 dias	10 ⁹ anos
FibIter	1/6 ms	1/3 ms	1/2 ms	3/4 ms	1,5 ms

Livro do
Ziviani

Tipos de Recursividade

- Se uma função F contiver uma referência explícita a si mesma, esta função é dita diretamente recursiva;
- Se $F1$ contiver uma referência a outra função $F2$, que por sua vez contém uma referência direta ou indireta a $F1$, então $F1$ é dita indiretamente recursiva;

Direta e Indireta

Direta:

```
imprime_numeros (n)
{
    if (n==1) printf ("\n 1");
    if (n>1)
    { printf ("\n %d",n);
      imprime_numeros (n-1);
    }
}
```

Indireta:

```
void imprime_numerosA (int n)
{
    printf ("Chamada da funcao A:");
};
if (n==1) printf (" 1\n");
if (n>1)
{ printf (" %d\n",n);
  imprime_numerosB (n-1);
}
```

```
void imprime_numerosB (int n)
{
    printf ("Chamada da funcao B:");
};
if (n==1) printf (" 1\n");
if (n>1)
{ printf (" %d\n",n);
  imprime_numerosA (n-1);
}
```

Atividade

1. Implemente uma função recursiva $\text{soma}(n)$ que calcula o somatório dos n primeiros números inteiros.
2. Escreva uma função recursiva que recebe como parâmetros um número real X e um inteiro N e retorne x elevado a N .

Resposta 1

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 int soma(int n)
5 {
6     int result = 0;
7     if (n <= 0)
8         return(n);
9     else
10    {
11        result = n + soma(n-1);
12        return result;
13    }
14 }
15
16 int main()
17 {
18     int valor;
19     printf("Digite um valor para o Somatorio: ");
20     scanf("%d", &valor);
21
22     printf("%d \n\n", soma(valor));
23
24     system("PAUSE");
25 }
```

Resposta 2

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 float pot(float X, int N)
5 {
6     if( N == 0 )
7         return 1;
8     else
9     {
10         if( N > 0 )
11             return (X * pot(X, N - 1));
12     }
13 }
14
15 int main()
16 {
17     int x, n;
18     printf("Digite um valor para X e um para N: ");
19     scanf("%d %d", &x, &n);
20
21     printf("Valor de %d elevado a %d : %f \n\n", x, n, pot(x,n));
22
23     system("PAUSE");
24 }
```

Dúvidas, Críticas ou Sugestões

diegobertolini@gmail.com

Atividades

1. Implemente uma função **recursiva** para computar o valor de 2^n

Atividades

```
recursividade.c x
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 float pot(float X, int N)
5 {
6     if( N == 0 )
7         return 1;
8     else
9     {
10         if( N > 0 )
11             return (X * pot(X, N - 1));
12     }
13 }
14
15 int main()
16 {
17     int x, n;
18     printf("Digite um valor para X e um para N: ");
19     scanf("%d %d", &x, &n);
20
21     printf("Valor de %d elevado a %d : %f \n\n", x, n, pot(x,n));
22
23     system("PAUSE");
24 }
25
```